Journal of Data-centric Machine Learning Research (2026) Submitted 8/26; Published

Most Uncertainty Doesn't Matter: An Execution-Grounded Dataset of Per-Token Consequence in Code Generation

Nafew Azim NAFEW.AZIM@NORTHSOUTH.EDU
Department of Electrical and Computer Engineering
North South University

Reviewed on OpenReview:

Editor:

Abstract

Selective distillation, RL with verifiable rewards, adaptive decoding, and token attribution all route compute or supervision by a model's *uncertainty*, measured as entropy, margin, loss, or KL divergence. Each assumes that uncertainty tracks *consequence*: whether committing differently at a token would have changed the outcome. No dataset tests that assumption, because testing it requires ground truth about single-token decisions.

Code execution supplies that ground truth. We release CONSEQ, a dataset of 143,786 token positions drawn from verified-correct model trajectories. Each position records what happens when the model's own second-choice token is substituted and the program is re-executed, across 14 models, 9 model families, 9 programming languages, and 3 benchmarks.

Uncertainty does not track consequence. Entropy performs at or below a random baseline in 24 of 28 model-language configurations, and none of 11 published selection criteria beats random on identical positions. Consequence is carried instead by *syntactic role*: numeric literals, operators, and property accesses. A selector defined by a regular expression over the token string, costing no inference, beats random in all 28 configurations, capturing 53.8–84.8% of consequence against a 26.4–37.3% baseline.

Two boundaries follow. Consequence composes on a fixed sequence, with 93.2% of programs surviving 1–32 simultaneous individually-harmless substitutions, but it does not transfer once decoding leaves that sequence: a perfect consequence oracle buys nothing over random position selection ($t = 0.00$). Per-token attribution is therefore valid for editing a fixed sequence and invalid for steering generation. And what counts as consequential is partly a property of the *language*. Static type checking converts silent logic faults into compile-time faults, by an amount that scales with checker strictness (Rust > Java > TypeScript > Go, in identical order for two model families), so semantic consequence is not comparable across typing disciplines. These results hold under two alternative counterfactual rules.

Keywords: datasets, code generation, token importance, credit assignment, program analysis

1 Introduction

1.1 The unexamined premise

Table 1 lists four literatures, the uncertainty signal each uses, what it routes, and the operational consequence of getting it wrong: wasted compute on supervised positions that did not matter, or failed rollouts that credit assignment could not locate. One assumption has never been tested against execution ground truth: that uncertainty tracks consequence.

Literature	Signal	Routes	Cost of a wrong signal
Selective dist.	entropy, margin, loss, KL	tokens to supervise	model trains on noise
RLVR	entropy, advantage, ent.-temp.	credit assignment	credit on wrong token
Adaptive dec.	entropy, varentropy, min-p	resample positions	waste decode budget
Token attrib.	gradient, attention, SHAP	explanatory tokens	explanation is wrong

Table 1: Four literatures, one untested assumption: uncertainty tracks consequence.

1.2 Why it has not been tested

Ground truth requires per-token counterfactual outcomes. Math tasks yield one bit at the end of a chain; open text requires a judge model. Code is the only domain where verification is objective, millisecond-cheap, and repeatable at any prefix. A perturbed program can also be executed without regenerating anything, which makes the headline measurement nearly free.

1.3 Contributions

1. **Conseq, the dataset:** 143,786 positions with deterministic counterfactual verdicts.
2. **The first audit** of 11 published selection criteria against execution ground truth on identical positions.
3. **A zero-cost selector** (syntactic role via regex) that beats all 11 criteria in every configuration measured.
4. **Two scope boundaries:** consequence composes on fixed sequences (93.2% survival) but does not transfer off-trajectory (oracle = random at decode time).
5. **Language-conditionality of consequence:** static typing shifts consequence from semantic (K^{sem}) to compile-time (K^{type}); the shift scales with checker strictness.

2 Related Work

Selective distillation and token-level data selection. Knowledge distillation (Hinton et al., 2015) was extended to sequences by Kim and Rush (2016) and to on-policy student generations by Agarwal et al. (2024) and Gu et al. (2024). A more recent line argues that not all positions deserve equal loss. Rho-1 (Lin et al., 2024) scores tokens by excess loss against a reference model and trains only on the top fraction; SelecTKD (Huang et al.,

2025) has the student propose and the teacher verify; Shen et al. (2026b) select by training-trajectory dynamics; Liu et al. (2025) weight by token-level uncertainty. Surveys (Song and Zheng, 2026; Tavor et al., 2026) organise these along position, class, and sample axes. Every criterion in this literature is validated the same way: it is inserted into a training pipeline and a downstream benchmark is read. That establishes a pipeline worked, not that a criterion identified the right positions. Conseq supplies the missing referee.

Credit assignment in RL with verifiable rewards. Policy-gradient methods (Williams, 1992; Schulman et al., 2017; Ouyang et al., 2022) and GRPO (Shao et al., 2024) assign a single trajectory-level advantage to every token. Recognising this as coarse, recent work modulates per-token credit: by entropy (Wang et al., 2025; Tan et al., 2025), by sequence-level likelihood (Lin et al., 2026), by Bayesian value recursion (Li et al., 2026), and by information-theoretic criteria (He et al., 2026); see also Shen et al. (2026a). These methods are evaluated by downstream reward, never against ground truth about which token actually mattered. Section 7 reports our attempt to supply such ground truth, and the negative result that follows for our own selector.

Decoding. Truncation and temperature schemes condition on uncertainty: nucleus sampling (Holtzman et al., 2020), typical decoding (Meister et al., 2023), truncation as desmoothing (Hewitt et al., 2022), and min-p (Nguyen et al., 2024). Our oracle experiment addresses this family directly.

Mutation testing. The instrument is mutation testing (Jia and Harman, 2011; Papadakis et al., 2019) with the roles inverted: that field holds the program fixed and scores the test suite, while we hold the tests fixed and score positions. We inherit its central hazard, equivalent mutants, and apply its remedies (Section 5). Fault localisation (Wong et al., 2016; Rafi et al., 2024) and program repair (Le Goues et al., 2019; Ye et al., 2021; Ye and Monperrus, 2023) likewise reason about which program locations are responsible for a failure, but from a fixed program instead of a model's decision distribution.

Attribution and interpretability. Causal tracing (Meng et al., 2022) and activation patching (Zhang and Nanda, 2024; Syed et al., 2024; Abdelsalam et al., 2026) corrupt and restore internal activations, measuring the effect on a log-probability. Gradient attribution (Sundararajan et al., 2017) and influence functions (Koh and Liang, 2017) attribute outputs to inputs or training data. Our intervention is on the *emitted token*, the object the four target literatures actually route, and our outcome variable is program correctness rather than a log-probability.

Code benchmarks. Problems come from HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), translated across languages by MultiPL-E (Cassano et al., 2023). Related efforts strengthen test suites (Liu et al., 2023), broaden library use (Zhuo et al., 2024), and probe self-invocation (Yu et al., 2024) and functional equivalence (Maveli et al., 2024); see also Yadav et al. (2024). We use these as a source of executable tasks, not as leaderboards.

Data-centric practice. We follow datasheet conventions (Gebru et al., 2021). The corrections log in Appendix C responds to evidence that data defects propagate silently through

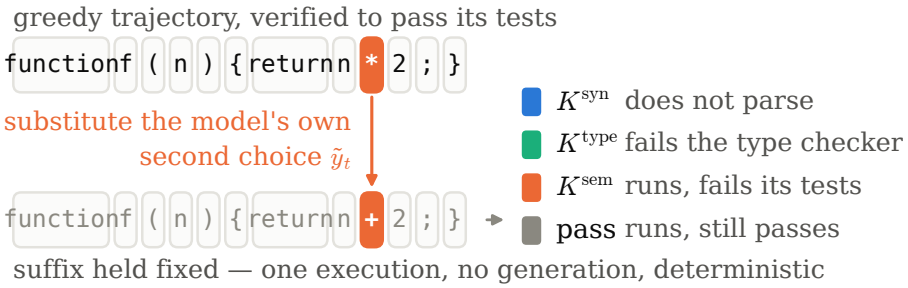

Figure 1: The measurement. One token of a verified-passing greedy trajectory is replaced by the model's own second choice; the suffix is held fixed and the program is executed once. Because nothing is regenerated, the verdict is a deterministic function of the substitution rather than a sample from a decoder. The four outcomes are ordered by how early the language rejects the program, which is what makes K^{sem} incomparable across typing disciplines (§6.5).

downstream results (Northcutt et al., 2021; Sambasivan et al., 2021; Kapoor and Narayanan, 2023) and to benchmark-construction practice in data-centric work (Mazumder et al., 2023).

3 The Instrument

3.1 Estimand

Let π be a model, q a programming task with test suite T, and $y = (y_1 \dots y_L)$ a greedy trajectory from $\pi(\cdot \mid q)$ that we retain only when $\text{pass}(y, T) = 1$. Attribution inside an already-broken program is meaningless, so every measurement in CONSEQ begins from a program that works.

At position t the counterfactual is the model's own second choice, $\tilde{y}_t = \arg\max_{v \neq y_t} \pi(v \mid q, y_{<t})$. This choice is deliberate: it bounds interpretation. CONSEQ measures the consequence of *the nearest decision the model could have made*, not the consequence of arbitrary corruption. A dataset built on random token replacement would mostly measure how easily programs break, a property of the language and not of the model's decisions. Section 6.6 tests both directions in which this rule could be wrong: alternatives that cannot parse, and alternatives further out in the distribution. The conclusions are unchanged.

3.2 The forced-splice arm (headline quantity)

The primary measurement holds the rest of the program fixed (Figure 1):

$$y^{\dagger} = y_{<t} \oplus \tilde{y}_t \oplus y_{>t}, \qquad K_t = \mathbf{1}[\text{pass}(y^{\dagger}, T) = 0].$$

This requires one execution and no generation. K_t is deterministic: there is no N, no confidence interval, no disattenuation. Every claim in Section 6 that rests on K_t rests on a complete measurement over the full position population.

3.3 Four-way verdict

Execution sorts each splice into four classes:

- K^{syn}: the program no longer parses.
- K^{type}: it parses but fails static type checking (languages with separate check phase: TS, Go, Java, Rust).
- K^{sem}: it parses, type-checks, and fails its tests.
- K^{timeout}: execution exceeded its budget.

The last class is excluded from consequence by construction, and the reason is empirical. Under CPU contention a slow toolchain times out on positions that are perfectly healthy. Java initially showed a 19.0% timeout rate against at most 1.4% elsewhere; counting those as consequence produced a K^{sem} of 26.9% that contradicted every other typed language. With timeouts excluded and worker counts matched to compile cost, the rate drops to 11.2%. Timeouts denote *unmeasured*, never *consequential*.

3.4 Free-continuation arms

The forced arm asks whether a choice mattered to the program. A second question is whether it mattered to the model: given the damaged prefix, can generation recover? We sample $N = 8$ free continuations from the factual prefix $y_{\leq t}$ and from the counterfactual prefix $y_{<t} \oplus \tilde{y}_t$ under shared seeds, giving pass rates $\hat{p}_F$ and $\hat{p}_C$, and define repairability $R_t = \hat{p}_C / \hat{p}_F$ and net damage $D_t = \hat{p}_F - \hat{p}_C$.

Resampling the factual arm is essential rather than decorative. Comparing the counterfactual against the original trajectory's pass = 1 would attribute ordinary resampling variance to the token. The matched design isolates the effect of the decision from the effect of continuing at all. In an early implementation this arm was mis-assembled and $\hat{p}_F$ read 0.09; since a correct prefix must usually be completable, that number was the signal that the measurement, not the model, was broken.

4 The Conseq Dataset

Every row carries its own provenance (model, family, parameter count, language, typing discipline, instrument version), so the file is self-describing and any subset is reconstructable without consulting a manifest. Problems come from HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) via MultiPL-E (Cassano et al., 2023), and from the original HumanEval for Python, which MultiPL-E does not target because Python is its source language. Models span nine families: Qwen2.5-Coder and Qwen3 (Hui et al., 2024; Yang et al., 2025), StarCoder2 (Lozhkov et al., 2024), DeepSeek-Coder (Guo et al., 2024), Granite Code (Mishra et al., 2024), Stable Code (Pinnaparaju et al., 2024), CodeGen (Nijkamp et al., 2023), InCoder (Fried et al., 2023), Phi-2 (Javaheripi et al., 2023), and SmolLM2 (Allal et al., 2025). We excluded models above roughly 3B parameters (Rozière et al., 2023; Zhu et al., 2024; Li et al., 2023) because 4-bit quantisation can alter the argmax and therefore corrupt the counterfactual token on which every measurement depends. A

Language	Typing	Problems	Positions	Consequential
JavaScript	dynamic	329	57,571	7,578
PHP	dynamic	262	21,447	3,169
TypeScript	gradual	178	21,373	1,954
Java	static	152	10,896	1,114
Perl	dynamic	173	9,788	3,102
Go	static	103	7,111	837
Python	dynamic	73	5,713	697
Rust	static	63	5,559	615
Ruby	dynamic	129	4,328	801
Total			**143,786**	

Table 2: Conseq coverage. Nine languages spanning dynamic, gradual, and static typing; 14 models across 9 families; 28 model-language configurations. Consequential counts are after artifact filtering (Section 5).

non-code-specialised sibling of one family is included to test whether the effect depends on code-specific training; it does not.

The intended use is direct. A researcher with a new token-importance heuristic scores the positions in Conseq and computes rank correlation against K^{sem} in seconds. This replaces the field's current practice: validate a criterion by training a model and reading a benchmark, which establishes that a *pipeline* worked (confounded by learning rate, data mixture, teacher choice, and budget) but says nothing about whether the criterion was correct.

5 Validating the Instrument

For a dataset paper this section is the central claim to credibility, and we report it in the main body rather than an appendix.

5.1 Three artifact classes (34% of raw K^{sem})

34.0% of raw K^{sem} positions are artifacts of the instrument, not decisions the model made.

Inconsistent-reference mutants (29.7% of raw K^{sem}) arise because substituting an identifier at one site while the forced suffix retains the original name breaks a *reference*, not a *decision*. Free continuation would simply have renamed consistently. These are detected by failure mode: a genuine keystone fails an assertion, whereas a rename throws `ReferenceError` in JavaScript, `NameError` in Python, `undefined:` in Go. Removing them sharpens the central result rather than weakening it, which is the signature of removed noise.

BPE fragment alternatives (4.3%) occur when the model's second choice is a sub-token of its first, such as `.length` → `.l`. This is tokenizer degeneracy, and not an alternative

Artifact	Share of raw K^{sem}	Detection
Inconsistent-ref. mutants	29.7%	`ReferenceError`/`NameError`, not `AssertionError`
BPE fragment alternatives	4.3%	`.length`→`.l`; both sides word-like
Equivalent mutants	0.0%	AST-normalised exact match

Table 3: Three artifact classes removed before analysis, as shares of raw K^{sem}. Removing them sharpens the entropy result (bottom-entropy tercile mass 47.3% → 53.1%), the signature of removed noise. The third class is worth its row precisely because its share is zero: equivalent mutants are 2.7% of *all* positions, and not one of them fails its tests, which is the check that the AST normalisation is doing what it claims.

the model was choosing between. The filter requires care: `<` → `<=` is also a prefix relation and is a genuine keystone, so a pair is flagged only when both sides are word-like.

Equivalent mutants are semantically identical after AST normalisation, the classical hazard of mutation testing. They are 2.7% of all positions and *zero* of raw K^{sem}: an AST-equivalent substitution cannot change a test outcome, so a non-zero share here would have indicated a defect in the normaliser instead of a property of the data. We report the class because that zero is a passed check, and not because it removes anything.

Artifact rates are tokenizer-specific, ranging from 19.9% to 46.2% across the six adequately-powered families and reaching 56% in the smallest, where the denominator is a few hundred positions. The filter is therefore re-validated per model family rather than assumed to transfer.

5.2 Invariants asserted at runtime

Each of the following is true by construction and is checked during data collection; a violation halts the run instead of producing plausible numbers:

1. A program valid enough to pass its own tests cannot have zero parseable single-token variants.
2. The factual continuation arm must pass at a high rate ($p_F \geq 0.8$).
3. Every arm in a decoding comparison must commit at identical counts.
4. Substituting at exactly one individually harmless position must leave the program passing ($m = 1$ composition $= 100\%$).
5. Timeout is excluded from K^{sem} (B9).
6. Linker failures are classified as unmeasured (B10).

Each invariant was added in response to a failure it would have caught. Over the course of this work, eleven measured results turned out to be instrument artifacts, two of which were fully retracted (B3: "consequence does not compose", which in fact it does, at 93.2%;

and B11, an apparent exception in the criterion audit that was our own sign error). The corrections: B1 (EOS decoded into program text, ∼70% trajectories silently discarded); B2 (continuation arms from wrong prefix, factual-arm pass 0.09); B3 (retracted, compositional claim); B4 (oracle handicapped against controls); B5 (bug-localisation injection artifact); B6 (Python requested from non-existent dataset); B7 (Python executed by JavaScript runtime: 0 consequential positions out of 2,937, every splice a parse error. The trajectory-validity check passed because it routed correctly, but the splice arm did not); B8 (Go executed as program, not test); B9 (timeouts counted as semantic, Java 19% timeout rate producing false $K^{\text{sem}} = 26.9\%$); B10 (Rust linker failure classified as syntax error); B11 (Rho-1 excess loss with reversed operands). Appendix C records each with the diagnostic signature that exposed it. One signature recurs through B1–B10: when positions that ought to be inert, such as punctuation and whitespace, appear consequential, the instrument is broken. In a working instrument those roles sit near 1%. B11 is the exception that marks the limit of the heuristic. Its numbers were unremarkable, and it surfaced only on reading the source paper's equation.

5.3 Determinism

Re-running a configuration in a fresh session reproduces position counts exactly ($5,097 \to 5,097$; $4,555 \to 4,555$; $3,613 \to 3,613$). Greedy trajectories, splice sets, and verdicts remain stable across sessions. Assembly refuses to merge rows collected under instrument versions whose columns changed meaning, and a coverage diff guards against silent problem-count regressions between versions.

Two pre-registered gates failed (Appendix F): the decoding gain prediction (G6) and an early compositional hypothesis (retracted after B3). We report both failures as negative results rather than re-analysing them.

6 Results

6.1 Entropy is at or below random

At a matched selection budget, ranking positions by entropy captures less execution-grounded consequence than selecting positions uniformly at random. This holds in 24 of 28 model-language configurations (Figure 2). The remaining four sit at parity, with margins under 4.3 points, and one of the four is an under-powered run flagged in the manifest.

The result survives three stress tests: problem-level clustering (within-problem paired test, $t = 11.95$, SE 0.012, $p < 10^{-19}$, positive in 67 of 74 problems), restriction to content tokens only, and removal of identifiers.

The magnitude varies. Entropy's shortfall ranges from 13.2 points below random at one extreme to 4.2 points above it at the other, with the four non-negative cases all within 4.3 points of chance. The *direction* is the robust claim; the size is family-dependent. The largest non-negative cell, DeepSeek on Python at +1.5 points, is an interaction and not a main effect of either factor: the same family is −7.0 on JavaScript and −7.2 on TypeScript, and the same language is −5.8 under Qwen (Figure 2).

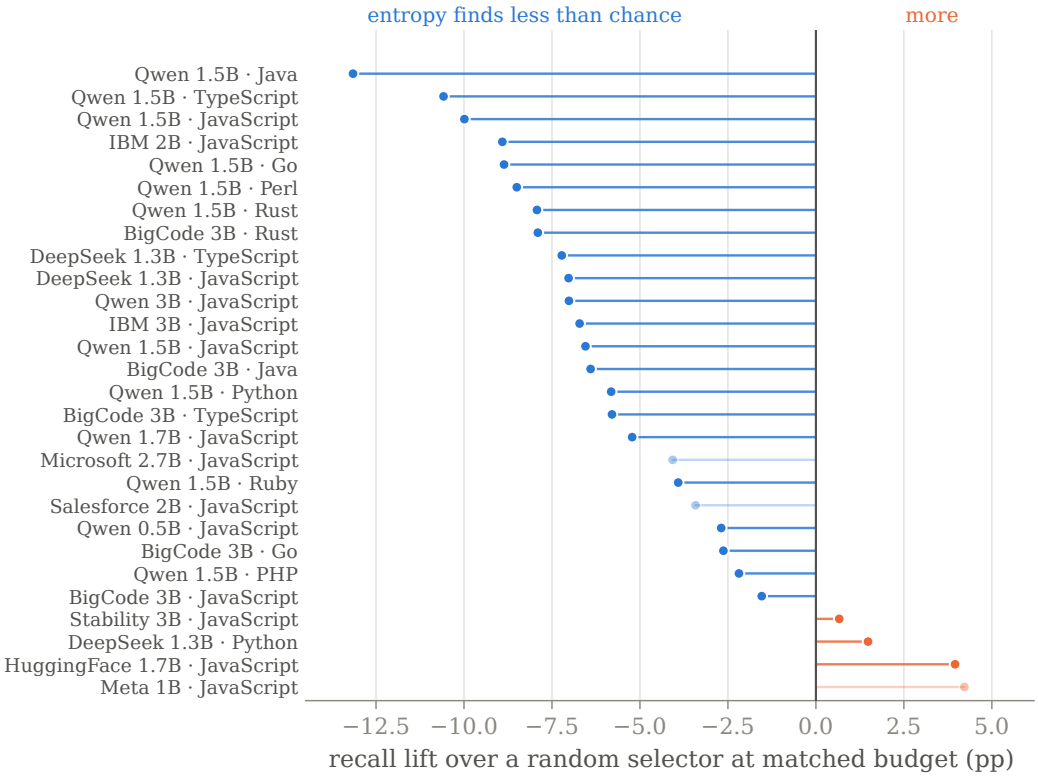

Figure 2: Entropy's recall of execution-grounded consequence, minus that of a random selector at the same budget, for every configuration in the release. Zero is chance. The selector is fit on half the problems and scored on the other half; budgets differ across configurations, which is why the comparison is expressed as a lift instead of a level. Faded rows are the three runs the manifest flags as under-powered (fewer than 25 retained problems); one of the four non-negative cases (InCoder-1B, 11 problems) is among them.

6.2 Consequence is carried by syntactic role

Numeric literals are simultaneously the lowest-entropy and most consequential role in every language measured. Operators and property accesses follow. Keywords, punctuation, and string contents are nearly inert. A selector defined by a regular expression over the token string, using no model call and no learned parameters, beats random in all 28 configurations. It captures between 2.1 and 2.5 times the random baseline (53.8–84.8% capture against a 26.4–37.3% baseline), and the pattern replicates in a second family and a second benchmark: the role selector recovers 71.3% of consequence in DeepSeek-Coder and 31.8% on MBPP, where entropy selects positions whose mean entropy is 0.162 and 0.252 respectively.

The substitutions that survive artifact filtering are the ones a reader would guess: a changed digit (`0`→`1`), a dropped compound assignment (`+=`→`=`), a loosened or tightened comparison (`>`→`>=` and `<`→`<=`), and an inverted equality test (`===`→`!==`). These are the constructs we predicted in advance.

Role	Consequence rate	Mean entropy
Numeric literal	62.4%	**0.114**
Identifier	24.3%	0.526
Property access	22.5%	0.233
Operator	20.8%	0.570
Keyword	6.3%	0.814
Punctuation	0.9%	0.699

Table 4: Consequence rate and mean entropy by syntactic role, pooled over the release. The two columns run in opposite directions: the roles that decide outcomes are the ones the model is most certain about, which is why an entropy-ranked selector systematically avoids them.

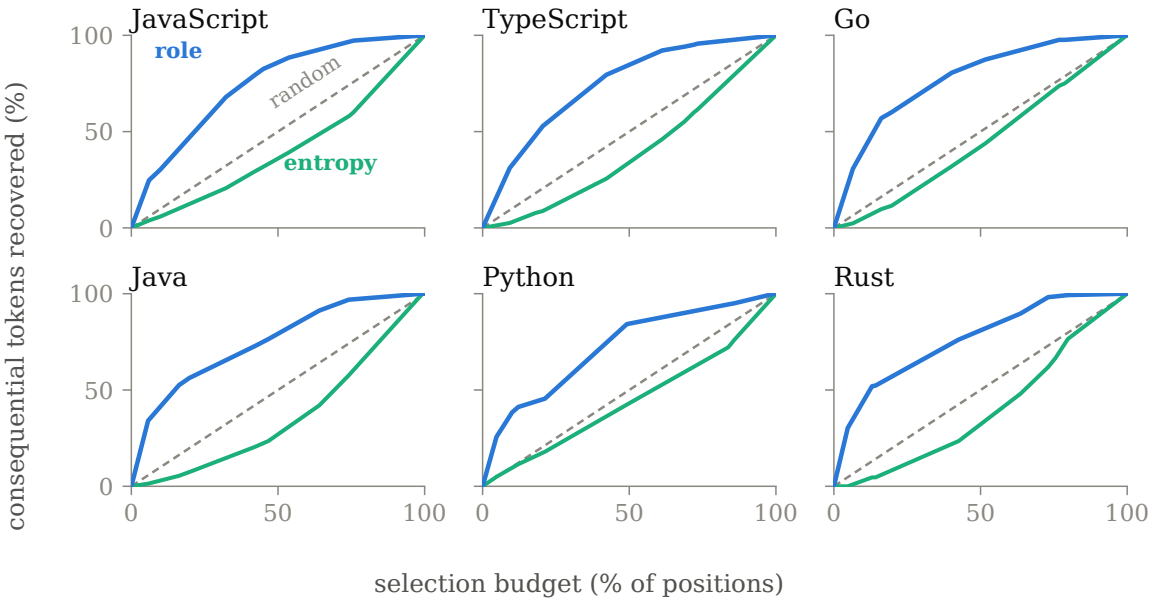

Figure 3: Budget–recall curves, held out. A selector that ranked positions perfectly would hug the top-left corner; the diagonal is random. The role selector is above the diagonal at every budget in every language, and entropy is below it at every budget in every language. The gap is not an artifact of the single operating point reported in Figure 2. Curves pool all models measured in each language.

A regular expression outperforms every learned criterion. We interpret this as evidence about the criteria, not the benchmark. Uncertainty-based criteria concentrate on positions where many choices are acceptable. Consequence is lowest at these positions by definition.

Criterion	ρ	CC@10	CC@25	Waste@25
Confidently-wrong* (Xu et al., 2026)	+0.000	9.0%	21.6%	86.8%
Random baseline	−0.003	8.5%	**23.5%**	85.6%
Position*	−0.034	5.7%	19.6%	88.0%
Excess loss (Lin et al., 2024)	−0.067	2.5%	12.9%	92.1%
Teacher–student KL (Hinton et al., 2015)	−0.094	8.5%	22.5%	86.3%
Expert–generalist delta*	−0.134	3.5%	15.4%	90.6%
Min-p width (Nguyen et al., 2024)	−0.158	4.1%	11.3%	93.1%
Margin*	−0.185	4.4%	13.1%	92.0%
Varentropy*	−0.190	4.6%	14.9%	90.9%
Negative log-likelihood*	−0.190	3.9%	13.6%	91.7%
Entropy (Wang et al., 2025)	**−0.196**	4.4%	**12.4%**	92.4%

Table 5: Eleven selection criteria scored on identical positions, each in the direction its own literature uses it. Rows marked * have a per-position form of ours: either a standard quantity used across a literature with no single canonical definition, or, where a paper is cited alongside, our adaptation of that paper's criterion to a per-position score. Appendix E gives every formula and every parameter we chose. ρ is Spearman correlation with K^{sem}; CC@b is the fraction of total consequence captured at budget b; waste is the fraction of the selected budget spent on positions where the choice did not matter. Random gives CC@$b \approx b$.

6.3 No published criterion beats random

The role selector's superiority raises a question: do the criteria it outperforms simply lack signal, or are they measuring something orthogonal to consequence? We tested all 11 selection criteria in use across these four literatures, scored on identical positions (3,697 positions, 62 problems), each in the direction its own literature uses it (entropy/NLL/KL/excess-loss select high; margin selects low; prefix methods select early).

Table 5 reports the audit results. Three observations follow.

First, no criterion beats random: the best CC@25 belongs to the random baseline. Second, entropy performs worst of the eleven, despite being the most widely deployed. Third, and most consequential for practice, ten of the eleven correlate *negatively* with consequence and would score *better if their scores were reversed*. A practitioner who selected the *lowest*-entropy tokens instead of the highest would, on average, capture more consequence. These signals are not merely uninformative; they point the wrong way.

One criterion deserves separate mention, because an earlier version of this paper reported it as an exception. We had implemented Rho-1's excess loss with the operands reversed, computing $L_{\text{RM}} - L_\theta$ where Lin et al. (2024) define $L_\Delta(x_i) = L_\theta(x_i) - L_{\text{RM}}(x_i)$ and select the top k% of it. Under our inverted sign the criterion appeared to be the only one positively correlated with consequence ($\rho = +0.067$); scored as published it is $\rho = -0.067$ and captures 12.9% of consequence at a 25% budget, well below the 23.5% a random selector achieves. The correction is recorded in Appendix C (B11) and it removes the single exception to the negative result rather than creating one.

Waste is uniformly severe: between 86% and 93% of every selection budget is spent on positions where the model's choice did not affect the outcome. Put differently, for every ten tokens a practitioner supervises using any current criterion, roughly nine contribute nothing.

6.4 Repairability is bimodal

Given a damaged prefix, the model either recovers completely or not at all.

Outcome	JS 1.5B	Python 1.5B	JS 3B
$R = 0$ committed error, never recovers	34.2%	38.1%	31.0%
$0 < R < 0.5$ mostly broken	11.1%	9.2%	9.4%
$0.5 \leq R < 1$ mostly repaired	17.1%	16.3%	20.9%
$R \geq 1$ fully repaired	37.7%	36.4%	38.8%
median R	0.62	0.57	0.67
positions	398	239	449
factual arm $\hat{p}_F$ (sanity)	0.86	0.91	0.89
control D on non-consequential	+0.054	+0.069	+0.009

Table 6: Repairability is bimodal: damage is either committed or fully absorbed, and the middle is thin. Replicated across two languages and a twofold change in scale. The control row is the check that the forced and free arms agree where the forced arm reports no damage.

Table 6 shows 34% of consequential positions are committed errors and 38% are fully repaired. Entropy again points in the wrong direction. Committed errors are 2.4 times more frequent in the bottom entropy tercile than the top. This directional failure appears on a second, independently measured quantity, which provides strong evidence that the central result is not an artifact of the consequence definition.

6.5 Consequence is language-conditional

Static type checking does not reduce the frequency at which a token choice matters. It changes when the error surfaces. Five dynamic languages span 24.7–37.3%. Four typed languages span 9.6–13.2%.

The effect is graded. Ranking typed languages by checker interception rate yields Rust > Java > TypeScript > Go. A second, independent model family (StarCoder2-3B) reproduces this ordering exactly. Rust possesses the strictest checker and intercepts the most. Two families agree on JavaScript's semantic consequence to within 0.1 percentage points (Qwen 24.7% versus StarCoder2 24.6%). This confirms the quantity is a property of the language and not of the model.

This bounds cross-language comparisons. K^{sem} alone is not comparable across type disciplines. Comparisons must use $K^{\text{sem}} + K^{\text{type}}$.

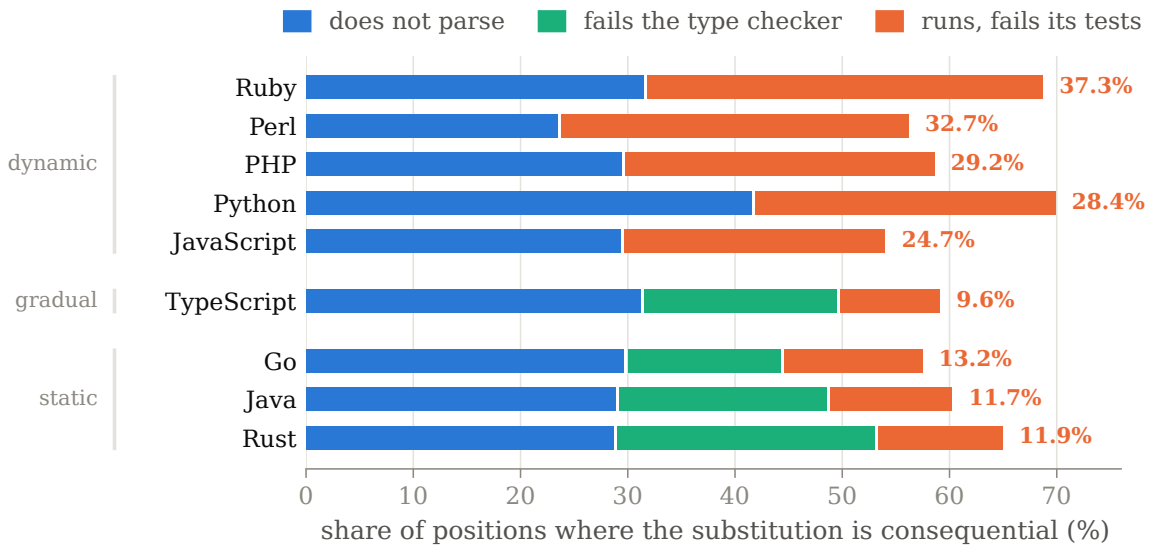

Figure 4: Where a wrong token surfaces, by the language's typing discipline. Bars decompose the consequential share into the stage that catches the substitution; the labelled value is K^{sem}, the faults that reach execution. Model, task set and instrument are held constant (one model, the same HumanEval problems translated across nine languages), so the grouping is attributable to the language. The dynamic and typed ranges do not overlap. Exact values in Table 12.

6.6 Robustness to the counterfactual rule

The estimand fixes one counterfactual per position: the model's own second choice. Two objections follow, and we measured both instead of arguing them.

The second choice is often illegal. If a large share of consequence were simply "the runner-up token does not parse", the finding would be about tokenisers and not about code. We re-ran Phase A with a *parse-preserving* rule: walk down the model's own ranked alternatives until one yields a parseable program, falling back to the second choice if none of the top ten does. Same model, same problems, same positions (Figure 5).

Syntax consequence was substitutable, not intrinsic: K^{syn} falls to roughly 5% in all three languages, and the residual equals exactly the rate at which no alternative in the top ten parses (5.0%, 4.3%, and 5.1% respectively). The mass moves down the pipeline instead of disappearing: to K^{sem} in JavaScript (+14.1pp) and to K^{type} in TypeScript (+11.3pp) and Go (+9.5pp). This is the language-conditionality result of Figure 4 in its cleanest form: once syntax is equalised across languages, the type checker is visibly the component absorbing the difference. The selector result is unchanged and larger. Held-out recall for ROLE, entropy, and random is 32.0%, 7.6%, and 13.8% on JavaScript; 49.0%, 11.2%, and 18.4% on TypeScript; and 51.4%, 8.1%, and 14.4% on Go. Entropy is below random in all

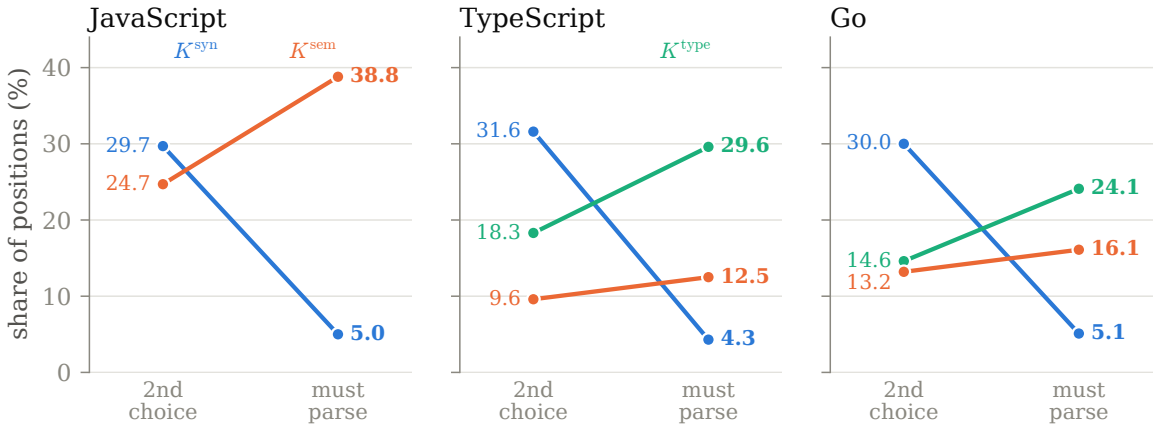

Figure 5: Requiring the counterfactual to parse does not remove consequence; it moves it downstream. Each panel is a matched position set (5,097 / 5,703 / 2,261), so the two ends of every line are the same positions measured under two substitution rules. The crossing is the result: what the second-choice rule scored as a syntax fault is, under a legal alternative, a semantic fault in JavaScript and a type fault in TypeScript and Go.

three. The median alternative rank is 2 (mean 2.51–2.60), so the two rules usually return the same token and the difference is carried by a minority of positions.

We report one confound instead of hiding it: parse-preserving selection biases toward common tokens, since rare alternatives more often break syntax. This pushes conservatively for the syntax claim, since it is what makes K^{syn} fall, and is neutral for the selector claim, which is scored within-rule.

The nearest alternative is not what a decoder samples. A decoder at temperature 1.0 draws from the whole distribution instead of the runner-up. We re-ran JavaScript with a temperature-sampled counterfactual (emitted token zeroed out) on the *same* 5,097 positions, giving a paired comparison instead of a two-sample one. Total consequence ($K^{\text{syn}} + K^{\text{sem}}$) rises from 54.4% to 58.2%, with K^{syn} moving 29.7%→31.2% and K^{sem} 24.7%→27.0%. Per-position agreement on the binary verdict is 90.1% (both consequential 51.3%, second-choice-only 3.1%, temperature-only 6.9%). The asymmetry is the expected one: a position that survives its nearest alternative sometimes fails a distant one, and the reverse is rare. **The second-choice estimand is a mild lower bound on consequence under realistic decoding, not a different quantity.** Role structure is unaffected: held-out ROLE 57.2% against entropy 15.6% at a 28.2% budget, with punctuation at 1.2%, the inertness invariant of §5.2.

6.7 Composition holds; off-trajectory transfer does not

Substituting the model's second choice at *every* individually harmless position at once, a mean of 23 simultaneous edits per program (range 1–32) each verified harmless in isolation,

leaves 93.2% of programs passing (Figure 6, left). Marginal per-token effects aggregate far better than one might expect, which is reassuring for any method that selects a *set* of tokens using individually computed scores.

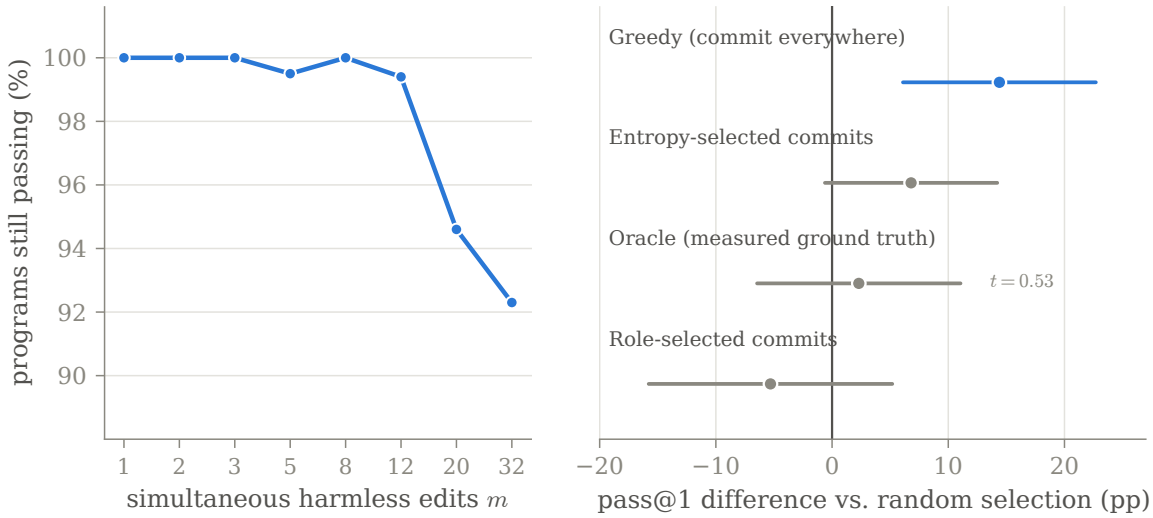

Figure 6: The boundary this dataset draws around its own use. *Left panel:* substituting the second choice at m positions that are each individually harmless, simultaneously. Marginal effects aggregate almost perfectly up to a dozen edits and degrade gently thereafter. $m = 1$ must be 100% by construction, and reading 66.8% is what exposed an earlier definitional error (Appendix C, B3). *Right panel:* decoding policies, named beside their intervals, at a matched commit budget, paired over 44 problems, as a difference against random position selection with 95% intervals. Only committing everywhere clears zero; a policy driven by measured ground truth does not. Exact values in Table 13.

The picture changes completely once the sequence is allowed to change. We compared five decoding policies that commit greedily at an identical number of positions per problem and sample elsewhere, differing only in *which* positions they commit at.

A policy driven by measured ground truth performed indistinguishably from one choosing positions at random (Figure 6, right; absolute pass@1 in Table 7). Both the role selector and entropy lose to random. Only committing everywhere helped, by 26 points.

We draw the boundary explicitly, because this is the most consequential claim this dataset makes about its own use. **Per-token attribution is valid for editing a fixed sequence and invalid for steering generation.** Consequence labels are properties of one trajectory; they compose while the surrounding tokens are held constant, and they stop applying once decoding leaves that path. No better predictor can rescue a decode-time method, because the ceiling itself sits at random.

Policy	pass@1	vs. random	t
Greedy (commit everywhere)	99.2%	+0.144	+3.50
Entropy-selected commits	89.4%	+0.068	+1.85
Oracle (ground truth)	84.8%	**+0.023**	**+0.53**
Random commits	82.6%	—	—
Role-selected commits	77.3%	−0.053	−1.02

Table 7: Decoding policies at matched commit budget (fraction 0.537), 44 paired problems. Arms differ only in *which* positions they commit at. A ground-truth oracle is statistically indistinguishable from random position selection; only committing everywhere helps.

7 Implications, and Their Limits

Selective distillation and token attribution: Applicable. Both assign scores to a fixed, already-generated sequence, the regime where composition holds. Table 5 shows that current criteria do not capture the quantity they target.

Adaptive decoding: Not applicable. Decoding steers generation. Our oracle experiment shows the position-selective decoding family fails, even given a perfect predictor.

RL credit assignment: Not applicable. Conseq measures local correctness on successful trajectories. RLVR credit assignment identifies where a failing rollout becomes unrecoverable. We binary-searched 76 failing rollouts to locate the fatal token.

Scorer	MRR	r@1	r@5	r@10
Token surprisal (NLL)	**0.306**	0.092	**0.579**	0.645
Entropy	0.184	0.039	0.342	0.605
Uniform (GRPO baseline)*	0.165	0.053	0.145	0.605
Margin	0.142	0.013	0.171	0.658
Syntactic role (ours)	**0.128**	0.053	0.132	0.289
Random	0.121	0.026	0.197	0.368

Table 8: Locating the token at which a failing rollout became unrecoverable, 76 cases. *Uniform is a constant scorer, so its ranking reflects tie-break order only; its apparent skill is the prior that fatal decisions occur early, not credit assignment.

The median fatal decision occurs at position 6 of a 40-token rollout. This indicates early structural planning failures, not local syntactic correctness.

On this task, the syntactic-role selector performs at random (0.128 MRR). Token surprisal (NLL) performs best (0.306 MRR). This is *provisional*: 76 cases, one model (Qwen2.5-Coder-1.5B), one language (HumanEval-JS), temperature-0.8 rollouts. NLL's advantage may partly reflect that sampled-off-policy tokens are both improbable and mistaken—the same confound flagged in the bug-localisation experiment, though less severe here since these are the model's own rollouts. Conseq evaluates local syntactic keystones. RLVR

evaluates structural planning. The two phenomena are disjoint. One interpretation consistent with the data: high entropy marks where exploration alters reasoning direction; low entropy marks where local correctness is decided.

8 Limitations

Conseq measures correct trajectories only, and therefore locates where correctness was *decided* rather than where failure *originates*. Trajectories are greedy; sampled behaviour is unmeasured except in the credit-assignment experiment and the temperature-sampled counterfactual of §6.6 (confined to JavaScript). Perturbations are single-token, and while they compose on a fixed sequence they do not transfer off it. The counterfactual robustness checks cover three languages and one model; whether the parse-preserving collapse of K^{syn} holds for every family is untested, though the mechanism, that most runner-up tokens are locally illegal, is not model-specific. The RLVR credit-assignment experiment (Section 7) uses 76 failing rollouts from one model (Qwen2.5-Coder-1.5B) on one benchmark (HumanEval-JS); the result bounds the transfer claim but does not generalise across models or languages. Coverage is uneven: JavaScript accounts for roughly 38% of positions, Ruby and Rust for 3–4% each. Retention varies sharply by model, since older and non-code-specialised models solve few problems, and three runs with fewer than 25 retained problems (CodeGen-2B-mono, Phi-2, InCoder-1B) are flagged in the manifest and excluded from headline claims. Problems are benchmark-derived, so idioms are benchmark idioms and not repository code. The eleven criteria in Table 5 are our implementations from published definitions. We checked each against its source paper's equation, which is how the inverted operand in B11 was found; the five rows marked * there are still our per-position reading of a family, not a transcription of one paper's named score, and where reference implementations exist they should be preferred over ours. This remains the most likely residual point of error in the audit, and it is now the only claim in the paper that a reader cannot re-derive from the released files.

9 Availability

Conseq is available at `https://github.com/nafew-azim/CONSEQ`, released under CC BY 4.0; the accompanying code is MIT. The release comprises the assembled dataset (`conseq.jsonl`), the per-run manifest, the raw per-run outputs from which it is assembled, the assembly pipeline, the nine per-language executors, the figure-generation scripts, and the datasheet of Appendix H.

Assembly is reproducible from the raw outputs by a single command, and refuses to merge rows whose column semantics differ across instrument versions. Licensing, hosting, and maintenance commitments, including the authors' statement of responsibility, are set out in Appendix H.

Reproducibility Statement

This paper introduces a dataset and instrument whose core measurements are deterministic: given the same model, task, and instrument version, the consequence label K_t is

reproducible without sampling noise. All 143,786 positions were measured under greedy decoding with fixed seeds; splice verdicts depend only on program execution and are independent of hardware, random state, or floating-point behaviour. The assembly pipeline (`assemble.py`), per-language executors, manifest, and all intermediate artefacts are released alongside the dataset. Three determinism checks (§5.3) confirm that re-running a configuration in a fresh session reproduces position counts exactly. The audit of 11 selection criteria (Section 6.3) was scored on identical position sets using published definitions, each checked against its source equation; where reference implementations exist they should be preferred (Section 8). The RLVR credit-assignment experiment (Section 7) uses 76 failing rollouts from one model and one language; we flag it explicitly as provisional.

Acknowledgments and Disclosure of Funding

We used a large language model as a coding and drafting assistant during this work: it wrote and debugged parts of the measurement and analysis code, and assisted in drafting and editing the manuscript. Every number reported here is regenerated from the released data by `verify.py`, which fails if any claim in the paper disagrees with the data, and the author takes full responsibility for all content.

Broader Impact Statement

This work provides a measurement instrument for testing assumptions that are widespread across four active research areas (selective distillation, RLVR, adaptive decoding, token attribution). The primary societal consequence is indirect: if the instrument is adopted, researchers will stop optimising criteria that point the wrong way, redirecting compute from supervised-noise positions to positions that actually matter. The dataset itself contains only programs and execution traces extracted from publicly available benchmarks (HumanEval, MBPP); it includes no user data, no personally identifiable information, and no content that could facilitate harmful generation. The code executors run in isolated sandboxes with no network access. The principal risk is misuse of the consequence labels to rank tokens by importance in security-sensitive code, where a wrong label could mask a vulnerability; this is mitigated by the corrections log (Appendix C), which documents every known instrument failure and its diagnostic signature.

References

Youssef Abdelsalam, Norman Peitek, Anna-Maria Maurer, Marvin Wyrich, and Sven Apel. A mechanistic lens on semantic conflicts: Using activation patching to understand LLM behavior. *arXiv preprint arXiv:2607.05587*, 2026.

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, et al. On-policy distillation of language models: Learning from self-generated mistakes. In *ICLR*, 2024.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, et al. SmolLM2: When smol goes big—data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, et al. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7), 2023.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, et al. InCoder: A generative model for code infilling and synthesis. *ICLR*, 2023. arXiv:2204.05999.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *ICLR*, 2024.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, et al. DeepSeek-Coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Yinhan He, Yaochen Zhu, Mingjia Shi, Wendy Zheng, Lin Su, et al. IAPO: Information-aware policy optimization for token-efficient reasoning. *arXiv preprint arXiv:2602.19049*, 2026.

John Hewitt, Christopher D. Manning, and Percy Liang. Truncation sampling as language model desmoothing. *Findings of EMNLP*, 2022.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *ICLR*, 2020.

Haiduo Huang, Jiangcheng Song, Yadong Zhang, and Pengju Ren. SelecTKD: Selective token-weighted knowledge distillation for LLMs. *arXiv preprint arXiv:2510.24021*, 2025.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Mojan Javaheripi, Sébastien Bubeck, et al. Phi-2: The surprising power of small language models. *Microsoft Research Blog*, 2023.

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 2023.

Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *EMNLP*, 2016.

Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions. In *ICML*, 2017.

Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019.

Raymond Li, Loubna Ben Allal, Yangtian Zi, et al. StarCoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

Yu Li, Rui Miao, Tian Lan, and Zhengling Qi. OPPO: Bayesian value recursion for token-level credit assignment in LLM reasoning. *arXiv preprint arXiv:2605.21851*, 2026.

Xingyu Lin, Yilin Wen, Du Su, Jinchang Hou, En Wang, et al. Token-level policy optimization: Linking group-level rewards to token-level aggregation via sequence-level likelihood. *arXiv preprint arXiv:2604.12736*, 2026.

Zhenghao Lin, Zhibin Gou, Yeyun Gong, Xiao Liu, Ruochen Xu, Chen Lin, Yujiu Yang, Jian Jiao, Nan Duan, and Weizhu Chen. Rho-1: Not all tokens are what you need. *arXiv preprint arXiv:2404.07965*, 2024.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *NeurIPS*, 2023.

Tingkai Liu, Ari S. Benjamin, and Anthony M. Zador. Token-level uncertainty-aware objective for language model post-training. *arXiv preprint arXiv:2503.16511*, 2025.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, et al. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.

Nickil Maveli, Antonio Vergari, and Shay B. Cohen. What can large language models capture about code functional equivalence? *arXiv preprint arXiv:2408.11081*, 2024.

Mark Mazumder, Colby Banbury, Xiaozhe Yao, et al. DataPerf: Benchmarks for data-centric AI development. *NeurIPS Datasets and Benchmarks*, 2023.

Clara Meister, Tiago Pimentel, Gian Wiher, and Ryan Cotterell. Locally typical sampling. In *TACL*, 2023.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *NeurIPS*, 2022.

Mayank Mishra, Matt Stallone, Gaoyuan Zhang, et al. Granite code models: A family of open foundation models for code intelligence. *arXiv preprint arXiv:2405.04324*, 2024.

Minh Nguyen et al. Turning up the heat: Min-p sampling for creative and coherent LLM outputs. *arXiv preprint arXiv:2407.01082*, 2024.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, et al. CodeGen: An open large language model for code with multi-turn program synthesis. *ICLR*, 2023. arXiv:2203.13474.

Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. Pervasive label errors in test sets destabilize machine learning benchmarks. *NeurIPS Datasets and Benchmarks*, 2021.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. *Advances in Computers*, 112:275–378, 2019.

Nikhil Pinnaparaju, Reshinth Adithyan, Duy Phung, et al. Stable code technical report. *arXiv preprint arXiv:2404.01226*, 2024.

Md Nakhla Rafi, Lorena Barreto Simedo Pacheco, An Ran Chen, Jinqiu Yang, Tse-Hsun, and Chen. SBEST: Spectrum-based fault localization without fault-triggering tests. *arXiv preprint arXiv:2405.00565*, 2024.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Nithya Sambasivan, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, and Lora Aroyo. "everyone wants to do the model work, not the data work": Data cascades in high-stakes AI. *CHI*, 2021.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Jianghan Shen, Siqi Luo, Yue Li, Jiyao Liu, Wanying Qu, et al. A first-principles derivation of LLM policy optimization: From expected reward to GRPO and its structural extensions. *arXiv preprint arXiv:2606.16733*, 2026a.

Zhanming Shen, Jiaqi Hu, Zeyu Qin, Hao Chen, Wentao Ye, et al. Training-trajectory-aware token selection. *arXiv preprint arXiv:2601.10348*, 2026b.

Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models. *arXiv preprint arXiv:2604.00626*, 2026.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *ICML*, 2017.

Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. In *BlackboxNLP*, 2024.

Hongze Tan, Zihan Wang, Jianfei Pan, Jinghao Lin, Hao Wang, et al. GTPO and GRPO-S: Token and sequence-level reward shaping with policy entropy. *arXiv preprint arXiv:2508.04349*, 2025.

Almog Tavor, Itay Ebenspanger, Neil Cnaan, and Mor Geva. Rethinking selective knowledge distillation. *arXiv preprint arXiv:2602.01395*, 2026.

Shenzhi Wang et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning. In *NeurIPS*, 2025. arXiv:2506.01939.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. A survey on software fault localization. *IEEE Transactions on Software Engineering*, 42(8):707–740, 2016.

Yuanda Xu, Hejian Sang, Zhengze Zhou, Ran He, Zhipeng Wang, and Alborz Geramifard. TIP: Token importance in on-policy distillation. *arXiv preprint arXiv:2604.14084*, 2026.

Ankit Yadav, Himanshu Beniwal, and Mayank Singh. PythonSaga: Redefining the benchmark to evaluate code generating LLMs. *arXiv preprint arXiv:2401.03855*, 2024.

An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

He Ye and Martin Monperrus. ITER: Iterative neural repair for multi-location patches. *arXiv preprint arXiv:2304.12015*, 2023.

He Ye, Matias Martinez, and Martin Monperrus. Neural program repair with execution-based backpropagation. *arXiv preprint arXiv:2105.04123*, 2021.

Zhaojian Yu, Yilun Zhao, Arman Cohan, and Xiao-Ping Zhang. HumanEval Pro and MBPP Pro: Evaluating large language models on self-invoking code generation. *arXiv preprint arXiv:2412.21199*, 2024.

Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models. *ICLR*, 2024.

Qihao Zhu, Daya Guo, Zhihong Shao, et al. DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931*, 2024.

Terry Yue Zhuo et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

Appendix A. Per-Run Manifest

Every run in the release, with the retention and consequence counts that determine whether it supports a claim. Runs marked † retain fewer than 25 problems and are excluded from headline claims; they are retained because coverage of older and non-code-specialised models is itself informative about where this instrument can be applied at all.

Run	Model	Family	Lang	Problems	Positions	Consequential
`o_scgo`	starcoder2-3b	BigCode	go	35	2,346	289
`o_scjava`	starcoder2-3b	BigCode	java	47	3,006	276
`o_v2sc`	starcoder2-3b	BigCode	js	57	4,555	646
`o_scrust`	starcoder2-3b	BigCode	rs	38	2,269	230
`o_scts`	starcoder2-3b	BigCode	ts	64	4,639	521
`o_v2ds`	deepseek-coder-1.3b-base	DeepSeek	js	49	3,613	429
`o_v2dspy`	deepseek-coder-1.3b-base	DeepSeek	py	48	2,776	285
`outdt`	deepseek-coder-1.3b-base	DeepSeek	ts	49	3,604	393
`o_smol`	SmolLM2-1.7B	HuggingFace	js	38	3,014	342
`o_g33`	granite-3.3-2b-base	IBM	js	47	3,565	530
`o_gr`	granite-3b-code-base	IBM	js	55	4,390	636
`o_incoder`	incoder-1B	Meta	js	11	237	24 †
`o_phi2`	phi-2	Microsoft	js	20	1,444	151 †
`o_v2nc`	Qwen2.5-1.5B	Qwen	js	58	3,286	494
`o_v205`	Qwen2.5-Coder-0.5B	Qwen	js	46	2,437	356
`o_mgo`	Qwen2.5-Coder-1.5B	Qwen	go	56	2,504	255
`o_v2go`	Qwen2.5-Coder-1.5B	Qwen	go	38	2,261	293
`o_mjava`	Qwen2.5-Coder-1.5B	Qwen	java	80	4,068	404
`o_v2java`	Qwen2.5-Coder-1.5B	Qwen	java	63	3,822	434
`o_v2mbpp`	Qwen2.5-Coder-1.5B	Qwen	js	218	11,670	1,150
`o_v2js`	Qwen2.5-Coder-1.5B	Qwen	js	77	5,097	784
`o_mphp`	Qwen2.5-Coder-1.5B	Qwen	php	189	15,863	2,178
`o_php`	Qwen2.5-Coder-1.5B	Qwen	php	73	5,584	991
`o_mpl`	Qwen2.5-Coder-1.5B	Qwen	pl	130	6,650	2,141
`o_perl`	Qwen2.5-Coder-1.5B	Qwen	pl	43	3,138	961
`o_v2py`	Qwen2.5-Coder-1.5B	Qwen	py	66	2,937	412
`o_rb`	Qwen2.5-Coder-1.5B	Qwen	rb	53	2,208	475
`o_mrb`	Qwen2.5-Coder-1.5B	Qwen	rb	76	2,120	326
`o_rust`	Qwen2.5-Coder-1.5B	Qwen	rs	59	3,290	385
`o_mts`	Qwen2.5-Coder-1.5B	Qwen	ts	90	7,427	503
`o_v2ts`	Qwen2.5-Coder-1.5B	Qwen	ts	76	5,703	537
`o_v23b`	Qwen2.5-Coder-3B	Qwen	js	88	6,057	914
`o_qwen3`	Qwen3-1.7B	Qwen	js	51	3,426	556
`o_codegen`	codegen-2B-mono	Salesforce	js	15	796	108 †
`o_stable`	stable-code-3b	Stability	js	53	3,984	458
Total					**143,786**	

Table 9: Per-run manifest. The run identifier is the output directory in the release, so every row of `conseq.jsonl` traces to the session that produced it; this is what distinguishes the several model-language pairs measured more than once (a second benchmark, or a counterfactual-rule variant). Runs marked † retain fewer than 25 problems, are flagged `underpowered` in the manifest file, and are excluded from headline claims.

Retention is the binding constraint on coverage; compute is not. A model must produce a verified-correct greedy trajectory before any position can be measured, so a model that cannot solve a task contributes nothing. InCoder-1B retains 11 of 161 HumanEval-JS tasks; Qwen2.5-Coder-1.5B retains 77. This asymmetry is a property of the measurement, not a sampling choice, and it means Conseq is systematically better populated for capable models.

Appendix B. Column Schema

Every row of `conseq.jsonl` is self-describing: provenance travels with the measurement, so any subset is reconstructable without consulting the manifest.

Column	Type	Meaning
`problem`	str	benchmark task identifier
`pos`	int	token index within the generated completion
`n_pos`	int	completion length in tokens
`tok`	str	the token the model emitted
`alt`	str	the counterfactual token (the model's second choice)
`verdict`	str	`pass` / `fail:<Class>` / `type_error:<Class>` / `parse_error` / `timeout`
`err`	str	error class parsed from the runtime's stderr
`K_syn`	bool	the substitution fails to parse
`K_type`	bool	parses, fails static type checking; always false where no separate phase exists
`K_sem`	bool	parses, type-checks, fails its tests
`K_timeout`	bool	execution exceeded its budget: *unmeasured*, never consequential
`entropy`	float	Shannon entropy of the model's distribution at this position
`margin`	float	p(top-1) $-$ p(top-2)
`suffix_len`	int	tokens remaining after this position
`equivalent`	bool	substitution is AST-equivalent after normalisation
`lang`	str	language identifier
`typing`	str	`dynamic` / `gradual` / `static`
`model`	str	model identifier
`family`	str	model family
`params`	str	parameter count
`iv`	int	instrument version (2 throughout this release)

Table 10: Schema of `conseq.jsonl`. Two columns carry conditions, not values, and are the most common source of misuse: `K_type` is language-conditional, and `K_timeout` marks missing data and not a negative result.

Two columns that are not what they look like. `K_type` is `false` both when a type checker passed the program and when no type checker exists, so any cross-language aggregate must condition on `lang`. `K_timeout` marks a measurement that did not complete;

treating those rows as "not consequential" inflates the pass rate, and treating them as consequential produced the contradiction recorded in B9.

Appendix C. Corrections

Eleven results in this project were believed before they were found to be artifacts of the instrument. We record them because a dataset paper's central claim is that its numbers mean what they say, and the credible form of that claim is a demonstration that the failure modes are known, and not an assertion that none occurred.

Each entry gives the symptom that exposed the defect. A pattern runs through the first ten and is worth stating as methodology: **every one was caught by a number being implausibly clean, and not by an error**. Nothing crashed. Each produced well-formed output that would have survived casual inspection. B11 is the exception, and it marks where that heuristic stops working.

B1. End-of-sequence token decoded into program text. The literal string `<|endoftext|>` reached the executed program, making every candidate a syntax error. This silently discarded roughly 70% of trajectories; retention rose from 18 to 77 problems on the fix. *Symptom:* an implausibly high parse-failure rate on programs the model had written competently. Caught only by logging *why* problems were skipped instead of counting how many.

B2. Continuation arms reconstructed from the wrong prefix. The model was conditioned on $y_{\leq t}$ but the program was reassembled from $y_{<t}$, dropping the token under test; the counterfactual arm dropped the substituted token entirely and measured nothing. *Symptom:* the factual arm passed 9% of the time when continuing from a known-correct prefix must usually succeed. Now asserted at runtime.

B3. "Consequence does not compose" (retracted). Reported as 0 of 44 programs surviving simultaneous substitution, and briefly treated as the project's most important finding. "Free" had been defined as *not* K^{sem}, which silently included artifact positions that genuinely break programs. *Symptom:* a *single* substitution at a supposedly free position passed only 66.8% of the time when it must pass approximately always. Under the corrected definition, $m = 1$ passes 100.0% and the aggregate is 93.2%, the opposite conclusion.

B4. Decoding oracle handicapped against its own controls. The oracle committed only at K^{sem} positions, leaving artifact-breaking positions inside its sampled set, and committed at 14.9% of positions while controls committed at 42.1%. *Symptom:* a ground-truth-driven policy performing worse than random, which is impossible if the comparison is fair. Arms are now asserted to commit at identical counts.

B5. Bug-localisation result driven by the injection procedure. Token surprisal appeared to localise injected bugs almost perfectly. Bugs were injected as the model's second choice, making the injected token the single improbable token in an otherwise top-1 sequence; surprisal was detecting the injection, not the bug. Reported but never claimed as a result.

B6. Python requested from a dataset that does not contain it. MultiPL-E translates *from* Python into other languages, so Python is its source and never a target. Python requires the original HumanEval, which uses a different schema.

B7. Python programs executed by the JavaScript runtime. The splice batch was routed through a helper that had never been given the language parameter. The trajectory-validity check *did* pass it, so problems were correctly retained and only the splice arm was wrong, which is why the run looked healthy. *Symptom:* 0 consequential positions of 2,937, with 100% parse errors. A program valid enough to pass its own tests cannot have zero parseable single-token variants; that invariant is now asserted and aborts the run.

B8. Go tasks executed as programs rather than tests. MultiPL-E ships Go tasks as Go *test* files; running them with `go run` compiles a package with no `main` and executes no assertions. *Symptom:* 0 of 154 problems retained. The Go self-check had passed because it used `os.Exit` instead of the `testing` package. **A self-check that does not reproduce the production format manufactures confidence**.

B9. Timeouts counted as semantic consequence. Java showed a 19.0% timeout rate against at most 1.4% elsewhere, because `javac` plus `java` per splice with eight workers on four cores is contention, not infinite loops. This produced a K^{sem} of 26.9% that contradicted every other typed language; excluding timeouts gives 11.2%. *Symptom:* punctuation at 21.8% and whitespace at 32.4% consequential, against roughly 1% and 5% in every other language. Worker counts and timeouts are now scaled per toolchain.

B10. Linker failure classified as a syntax error. Rust splices were sorted by diagnostic shape: a bare `error:` meant syntax, `error[E…]` meant a type or borrow fault. A `rustc` linker failure carries neither form, so environment failures such as a missing system linker under contention were being recorded as K^{syn}. The symptom was a Rust syntax rate that moved with machine load instead of with the substitution. Linker failures are now routed to `timeout`, the class reserved for unmeasured positions.

B11. Rho-1's excess loss implemented with reversed operands. Lin et al. (2024) define the excess loss as $L_\Delta(x_i) = L_\theta(x_i) - L_{\text{RM}}(x_i)$, the current model's loss minus the reference model's, and Selective Language Modeling trains on the top k%. Our implementation computed $L_{\text{RM}} - L_\theta$, which is the same quantity negated, so scoring it in the "select high" direction selected precisely the tokens the method discards. *Symptom:* excess loss was the sole criterion of eleven with a positive correlation, and the sole one that did not improve when reversed: an isolated exception in an otherwise uniform table, which is a shape worth distrusting. Scored as published, ρ moves from $+0.067$ to -0.067 and CC@25 from 23.2% to 12.9%. The selftest now asserts the sign as well as the zero: excess loss must vanish when reference and current model agree, *and* be positive when the current model fits the token worse. Unlike B1–B10 this defect was found by reading the source paper, not by an implausible number, which is why the criterion definitions are now stated as formulas in Appendix E instead of described in prose.

The diagnostic that generalises. In B3, B7, and B9 the tell was identical: positions that ought to be inert (punctuation, whitespace, formatting) appearing consequential. In a working instrument these sit near 1%. We recommend the check to anyone extending

this work: **if inert tokens look consequential, the instrument is broken, not the model.**

Appendix D. Language Executors

Each language requires its own execution contract, and three of the nine broke a structural assumption that the previous ones had shared. We document them because anyone extending Conseq will meet the same wall: **MultiPL-E's `tests` field is not uniformly self-executing**.

Lang	Parse check	Execute	Reference-error class (artifact)
JavaScript	`node --check`	`node`	`ReferenceError`, `TypeError`
TypeScript	`tsc` TS1xxx	`tsc` → `node`	`Cannot find name`
Python	`compile()`	`python3`	`NameError`, `AttributeError`
Ruby	`ruby -c`	`ruby`	`NameError`, `NoMethodError`
PHP	`php -l`	`php`	`Error`, `ArgumentCountError`
Perl	`perl -c`	`perl`	`Undefined subroutine`, `Global symbol`
Go	`gofmt -e`	`go test`	`undefined:`, `declared and not used`
Java	`javac`	`java -ea`	unresolved symbol
Rust	bare `error:`	`rustc` → binary	`E0425`, `E0433`, `E0412`

Separating syntax from types. Four languages permit a parse check independent of type checking, which is what makes K^{type} measurable. `gofmt -e` parses without type-checking; `tsc` error codes separate TS1xxx (syntax) from TS2xxx (type); `rustc` emits bare `error:` for syntax and `error[E....]` for type and borrow failures. The remaining five languages have no such phase and K^{type} is always false for them.

Harness idiosyncrasies. Go tasks are test files requiring `go test`. Perl tasks define `testhumaneval` without invoking it and depend on `Test::Deep`. TypeScript tasks declare `require` themselves because `tsc` has no Node type declarations. Java requires `-ea` or assertions are silently skipped and every program passes.

Environment failures are not code properties. A `rustc` linker failure carries no error code and would otherwise be classified as a syntax error, making a language appear entirely unparseable on a machine without a working linker. Such failures are classified as unmeasured.

Appendix E. Criterion Definitions

All eleven criteria are functions of distributions already computed during trajectory generation, so the marginal cost of the audit is two additional forward passes per trajectory, one for the reference model behind excess loss and one for the teacher behind divergence, with no extra generation.

Each criterion is scored in the direction *its own literature* uses it: entropy, negative log-likelihood, excess loss, and divergence select high values; margin selects low values, since

low margin denotes uncertainty; prefix methods select early positions. Scoring a criterion in the opposite direction to its intended use would be a straw man, and Table 5 reports separately whether a criterion would perform better reversed. Eight of eleven would.

Because an error in one of these definitions produced correction B11, we state all eleven as formulas instead of describing them. p is the student's next-token distribution at the position, y_t the emitted token, H its entropy, L the trajectory length.

#	Criterion	Score at position t	Selects
P0	Random	$u \sim \mathrm{Unif}(0,1)$	—
P1	Entropy	$H = -\sum_v p_v \log p_v$	high
P2	Margin	$p_{(1)} - p_{(2)}$	low
P3	Negative log-likelihood	$-\log p_{y_t}$	high
P4	Excess loss	$L_\theta(y_t) - L_{\mathrm{RM}}(y_t)$	high
P5	Teacher–student KL	$\sum_v p_v^{\mathrm{T}}(\log p_v^{\mathrm{T}} - \log p_v)$	high
P6	Expert–generalist delta	$\frac{1}{\lvert V\rvert}\sum_v \lvert\log p_v^{\mathrm{T}} - \log p_v^{\mathrm{G}}\rvert$	high
P7	Position	t/L	low (early)
P8	Confidently-wrong	$\mathbf{1}[H < 0.5 \ \wedge \ \arg\max_v p_v \neq y_t]$	high
P9	Varentropy	$\sum_v p_v(-\log p_v - H)^2$	high
P10	Min-p width	$\lvert\{v : p_v \geq p_{\mathrm{base}}\, p_{\max}\}\rvert$	high

Table 11: The eleven criteria as implemented. P4 follows Lin et al. (2024) eq. 3; the operand order is the subject of correction B11. P10 follows Nguyen et al. (2024) eq. 1 with $p_{\mathrm{base}} = 0.1$, within the range that paper uses (0.05–0.3) but a single point in it. P5 uses the forward KL from teacher to student, the direction used in knowledge distillation. P6 is our operationalisation of the delta-teacher family as a mean absolute log-ratio over the vocabulary; the underlying literature applies the log-ratio to a candidate token during decoding rather than aggregating it into a per-position score, so this row is a reasonable reading and not a transcription. P8 follows Xu et al. (2026), whose criterion is low student entropy together with high teacher–student divergence, meaning the student is confidently wrong. We approximate that term by whether the student's argmax differs from the emitted token, and the entropy threshold of 0.5 nats is a parameter we chose. Rows marked * in Table 5 are the ones whose per-position form is ours, not a single paper's.

Roles of the four models. The student is the model whose trajectory is measured (Qwen2.5-Coder-1.5B); the reference model for P4 is Qwen2.5-0.5B; the teacher for P5 and P6 is Qwen2.5-Coder-3B; the generalist for P6 is Qwen2.5-1.5B. All four share a tokenizer, without which the vocabulary-aligned criteria would not be computable.

Consequence Capture at budget b is

$$\mathrm{CC@}b = \frac{\lvert\{\, t \in \text{top-}b\% \text{ by criterion} \ : \ K_t^{\mathrm{sem}} = 1 \,\}\rvert}{\lvert\{\, t \ : \ K_t^{\mathrm{sem}} = 1 \,\}\rvert},$$

so a random selector gives CC@$b \approx b$ and the ground-truth selector gives the maximum attainable. This is the operative metric: it states directly how much of the available signal a criterion buys at the budget practitioners actually use.

Appendix F. Pre-Registration and Outcomes

Gates and kill criteria were fixed in a design document before measurement began. We report each with its outcome, including the two that failed, because a dataset whose analysis decisions were made after seeing the data is worth less than one whose were not.

Gate	Outcome	Note
G1 instrument validity	passed	after eleven corrections (Appendix C)
G2 signal non-degenerate	passed	consequential fraction 10–30% across languages
G3 orthogonality	passed	pre-registered as unfalsifiable: either sign was publishable
G4 repairability structured	passed	bimodal, replicated across languages and scale
G5 consequence predictable	passed	syntactic role, no learned parameters
G6 decoding gain	**failed**	reported in full; the family is closed

Two pre-registered predictions were wrong and are reported as such. The consequential fraction was predicted to *fall* with model scale; it is flat at 15–16% across a sixfold parameter range. And an early hypothesis that consequence would not compose was retracted after the definitional error recorded in B3.

G6 deserves emphasis. The pre-registered response to its failure was to report it as a negative result instead of searching for a configuration in which it succeeded. We followed that, and the oracle experiment of §6.7 is the result.

Appendix G. Exact Values for the Figures

Figures 4 and 6 are drawn from the tables below. One script produces the figure and the table from the same computation, so the two cannot disagree.

Language	Discipline	K^{syn}	K^{type}	K^{sem}
Ruby	dynamic	31.8%	—	**37.3%**
Perl	dynamic	23.8%	—	**32.7%**
PHP	dynamic	29.7%	—	**29.2%**
Python	dynamic	41.8%	—	**28.4%**
JavaScript	dynamic	29.7%	—	**24.7%**
Go	static (structural)	30.0%	14.6%	**13.2%**
Java	static (nominal)	29.2%	19.7%	**11.7%**
Rust	static (affine)	29.0%	24.3%	**11.9%**
TypeScript	gradual	31.6%	18.3%	**9.6%**

Table 12: Semantic consequence by typing discipline (Qwen2.5-Coder-1.5B). The dynamic and typed ranges do not overlap. K^{type} is defined only where a type-check phase is separable from parsing. StarCoder2-3B reproduces the strictness gradient: Rust 29.1%, Java 21.8%, TypeScript 18.8%, Go 12.8% (same ordering).

Simultaneous edits m	1	2	3	5	8	12	20	32
Pass rate	100.0%	100.0%	100.0%	99.5%	100.0%	99.4%	94.6%	92.3%

Table 13: Dose-response for composition. Program pass rate after substituting the second choice at m individually harmless positions simultaneously. $m = 1$ must be 100% by construction and is the sanity check that the free set is correctly defined; an earlier version read 66.8% and was diagnostic of a definitional error (Appendix C, B3).

Appendix H. Datasheet for Conseq

Following Gebru et al. (2021). Answers are specific to the release described in this paper (instrument version 2, 143,786 positions).

Motivation

For what purpose was the dataset created? To test, rather than assume, that a model's uncertainty at a token tracks whether that token's value changes the outcome. Four literatures route compute or supervision on that assumption; none had ground truth against which to check it. The gap is not conceptual but empirical: the counterfactual "what if the model had committed differently here" is unobservable in open-ended text, and observable in code only because a test suite adjudicates it.

Who created it and who funded it? Stated in the acknowledgements; DMLR review is single-blind and the author list is not withheld.

Composition

What do the instances represent? One instance is one *token position* in one verified-correct greedy trajectory produced by one model on one benchmark task, together with the execution verdict obtained when the model's own second choice is substituted at that position. An instance is therefore a measured counterfactual, not a piece of source text.

How many instances are there? 143,786, spanning 14 models, 9 model families, 9 programming languages, 3 benchmarks, and 28 model-language configurations. Per-language and per-run breakdowns are in Table 2 and Appendix A.

Is it a sample or the complete set? Complete, conditional on retention. Every position of every retained trajectory is measured; there is no position-level sampling. Retention is not random, since a trajectory enters only if it passes its tests, so the dataset is systematically better populated for more capable models. This is a property of the estimand (attribution inside an already-broken program is meaningless), not a sampling decision, and it is the dominant selection effect in the data.

What data does each instance consist of? The columns of Table 10: task and position identifiers, the emitted token and its alternative, the four verdict flags, the model's entropy and margin at that position, provenance (model, family, parameter count, language, typing discipline), and the instrument version.

Is there a label? Yes: the four-way verdict, of which K^{sem} is the primary target. It is a deterministic function of the substitution, not an annotation.

Is any information missing? K^{type} is undefined for the five languages whose toolchains do not separate type checking from parsing, and is stored as false there instead of as a distinct missing value; consumers must condition on language. Timed-out executions (K^{timeout}, 0.3–1.4% by language) are genuinely missing measurements and are excluded from consequence by construction, not counted as negative.

Are there errors, noise, or redundancies? Three classes of instrument artifact are identified and flagged instead of silently removed (§5.1); they account for 34.0% of raw K^{sem} positions and their rate is tokenizer-specific (19.9–46.2% across the six adequately-powered families). Ten measurement errors found during development are documented in Appendix C, including the two whose published claims were retracted.

Does it contain confidential or offensive content, or identify people? No. Instances derive from HumanEval and MBPP, both public benchmark suites of small self-contained programming tasks. There is no user data, no personally identifiable information, and no free text beyond programming-language tokens.

Collection

How was the data acquired? By direct measurement, with nothing observed or inferred. For each task the model generates greedily; trajectories that fail their tests are discarded. For each position of a retained trajectory, the model's second-choice token is spliced in, the suffix is held fixed, and the program is executed once against its own test suite.

What mechanisms were used? Open-weight models under `transformers` at `float16` on a single T4 GPU; nine language toolchains for execution (Appendix D). No human annotation is involved at any stage.

Over what timeframe? Measurements were collected over a single development cycle; every row carries an instrument version so that rows collected under different semantics cannot be pooled.

Were ethical review processes conducted? Not applicable: no human subjects, no personal data.

Preprocessing, cleaning, labelling

Was preprocessing done? Trajectory retention (pass-or-discard) and artifact classification. Artifact flags are stored as columns; nothing is deleted, so an analyst who disagrees with our taxonomy can recompute from the raw verdicts.

Is the raw data available? Yes. Per-run outputs are released alongside the assembled file, and `assemble.py` reconstructs the release from them in one command.

Uses

What tasks is it suitable for? Scoring and comparing token-selection criteria against execution-grounded ground truth; studying how consequence distributes over syntactic role; measuring how a language's type discipline relocates faults.

What should it *not* be used for? Two uses are contraindicated by our own measurements. It should not be used to justify a decode-time position-selection policy: consequence labels are properties of a fixed trajectory and stop applying once decoding leaves it, and a perfect oracle is statistically indistinguishable from random position selection (§6.7). And K^{sem} should not be compared across languages of different typing disciplines without adding K^{type}; the two ranges are disjoint for reasons that are a property of the toolchain and not of the model.

Is there anything that might cause unfair treatment? The retention asymmetry means claims supported by this dataset are better evidenced for capable, code-specialised models than for smaller or general-purpose ones. Three runs fall below 25 retained problems and are flagged in the manifest and excluded from headline claims.

Distribution, licensing, and maintenance

How is it distributed? As a single newline-delimited JSON file with its manifest, assembly pipeline, per-language executors, and this datasheet. The access URL is given in the Availability section.

Under what license? The dataset is released under CC BY 4.0 and the accompanying code under the MIT license. Instances derive from HumanEval (MIT), MBPP (CC BY 4.0), and MultiPL-E (MIT); all three permit redistribution of derived work with attribution, and all three are cited. Model weights are not redistributed, only measurements taken with them, and each model is cited to its own release.

Author statement of responsibility. The authors bear all responsibility in case of violation of rights and confirm that the data license permits the release described here.

Who maintains it, and how will errors be handled? The authors. The corrections log (Appendix C) is the maintenance mechanism as much as the record: every entry names the symptom that exposed the defect, so a consumer who observes the same symptom can identify the cause. Corrections are released as new instrument versions instead of silent edits, and `assemble.py` refuses to merge rows whose column semantics differ across versions. That is the guarantee that a future correction cannot retroactively contaminate this release.

Will it be updated? Additional model-language configurations can be added at roughly 0.5 GPU-hours each. Any change to column semantics increments the instrument version and quarantines incompatible rows instead of pooling them.

Reproducibility Checklist

1. **Claims.** The five contributions in §1.3 match the results in §6 and the scope boundaries in §6.7 and §7. Two claims made in an earlier version of this work were narrowed by our own measurements and the narrowing is stated in §7 rather than removed.

2. **Assumptions.** The estimand (§3.1) fixes the counterfactual to the model's second choice; the bounded-interpretation claim is stated there and tested in §6.6 under two alternative counterfactual rules. K_t is deterministic (no sampling noise), stated in §3.2.

3. **Theory.** N/A — the paper is empirical; no theorems are stated.

4. **Reproducibility.** The assembly pipeline, per-language executors (Appendix D), manifest (Appendix A), and figure scripts are released. Core measurements are deterministic (greedy decoding, fixed seeds); three determinism checks confirm exact reproducibility (§5.3).

5. **Data and Code.** The dataset (`conseq.jsonl`), the raw per-run outputs, the manifest, the assembly code, and the per-language executors are released together (§9). Single-command assembly from raw outputs. Column semantics are documented in Appendix B.

6. **Experimental Details.** No training is performed. Model identifiers, parameter counts, tokenizers, and problem sources are documented in §4 and in the per-run manifest.

7. **Statistical Significance.** The consequence label K_t is deterministic (§3.2), so the headline quantity carries no sampling error. Where sampling does enter—the free-continuation arms, the decoding experiment, the credit-assignment experiment—intervals or test statistics are reported. The entropy result survives a within-problem paired test ($t = 11.95$, $p < 10^{-19}$, §6.1). The RLVR experiment is flagged as provisional with $n = 76$ (§7).

8. **Compute Resources.** The forced-splice pass costs roughly 0.5 GPU-hours per model-language configuration on a single T4; the full release is on the order of 20 GPU-hours. Assembly requires a CPU machine with the nine language runtimes installed.

9. **Ethics.** The dataset contains only programs and execution traces derived from public benchmarks. No human subjects, no personally identifiable information, no content that could facilitate harmful generation. Executors run without network access.

10. **Broader Impact.** Stated before the references, as required by the venue.

11. **Licenses.** The dataset is released under CC BY 4.0 and the code under MIT. Source benchmarks (HumanEval, MIT; MBPP, CC BY 4.0; MultiPL-E, MIT) and every model used are cited to their own releases; model weights are not redistributed. Terms are set out in Appendix H.

12. **Assets.** A datasheet following Gebru et al. (2021) is given in Appendix H; a per-run manifest with provenance in Appendix A; and a corrections log with the diagnostic signature of each defect in Appendix C.